



Generación autónoma de microservicios

Del dolor a la verificación real: arquitectura Spec-Driven, orquestación de agentes y propuesta SOTA de auto-mejora

1. El Problema: 0 de 15 verificadas y la ilusión de la IA.

La adopción masiva de asistentes de código acelera la escritura de sintaxis, pero traslada el 76% del tiempo a depuración y genera deuda técnica acelerada.

EL DOLOR REAL

0 de 15 Verificadas

- El sistema generaba código con apariencia limpia y profesional.
- Al ejecutar pruebas herméticas, **ningún servicio compilaba**.
- Bloqueo real: defecto de montaje Docker y dependencias sin aislar.

LA ILUSIÓN DE LOS ASISTENTES

Fragmentación Ciega

- Copilot y Cursor asisten a nivel de archivo o función aislada.
- Nadie gobierna la coherencia entre capas, contratos ni dependencias.
- **76% del tiempo de ingeniería** se gasta en depurar y parchar.

LA CAUSA RAÍZ

Falta de Gobernanza

- Generar código sin compuertas herméticas acumula deuda técnica.
- No falta capacidad en el LLM; falta **diseño por especificación**.
- La solución exige que la especificación mande y verifique antes de entregar.

Principio rector: La promesa de la IA no es generar más líneas por minuto, sino entregar software corporativo verificado, auditable y listo para producción.

2. Solución y Alcance: Spec-Driven Design.

El modelo no inventa la arquitectura: la especificación formal es la única fuente de verdad inmutable.

Nivel de Especificación	Prescripción Contractual
Constitution (4 capas)	Dominio, Servicio, Infraestructura, API. Prohibición estricta de acoplamiento cruzado y llamadas circulares.
19 Contratos OpenAPI & BDD	Esquemas canónicos de datos, DTOs inmutables (records), códigos HTTP y escenarios Given-When-Then.
Lista blanca de dependencias	Librerías corporativas homologadas y versiones congeladas en POM raíz. Cero dependencias sorpresa.
Alcance tecnológico	Microservicios Java 21 LTS, Spring Boot 3.x, Spring Data JPA, Mockito, perfil hermético offline.

ARQUITECTURA DESACOPLADA

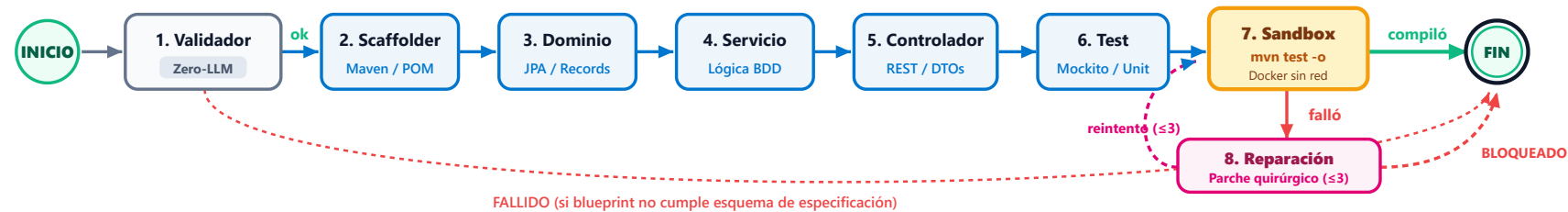
5 Subsistemas Modulares

- 1. Web Studio (React/Vite): 10 fases del ciclo de vida y streaming SSE en vivo.
- 2. API Gateway (FastAPI): Gestión de sesiones, persistencia y control de ejecución.
- 3. Motor Agéntico (LangGraph): Grafo coordinado de 8 agentes especializados con memoria.
- 4. Sandbox Hermético (Docker): mvn test -o --network none (contenedor sellado).
- 5. Observabilidad: Registro forense de costes y tokens en SQLite y MLflow.

Garantía innegociable: El LLM actúa estrictamente como un compilador de especificación a código Java ejecutable. Si no compila en sandbox hermético, **no se entrega**.

3. Funcionamiento: Orquestación LangGraph.

Grafo coordinado de 8 agentes con segregación de responsabilidades, compuertas deterministas y ciclo de reparación reflexivo acotado.



5 AGENTES ESPECIALIZADOS LLM

Especialistas de Dominio Técnico

- **1. Scaffolder:** Estructura Maven y dependencias de lista blanca.
- **2. Dominio:** Entidades JPA, records inmutables, repositorios.
- **3. Servicio:** Lógica transaccional y reglas de negocio BDD.
- **4. Controlador:** Endpoints REST y @RestControllerAdvice.
- **5. Test:** Cobertura unitaria y de integración mapeada a BDD.

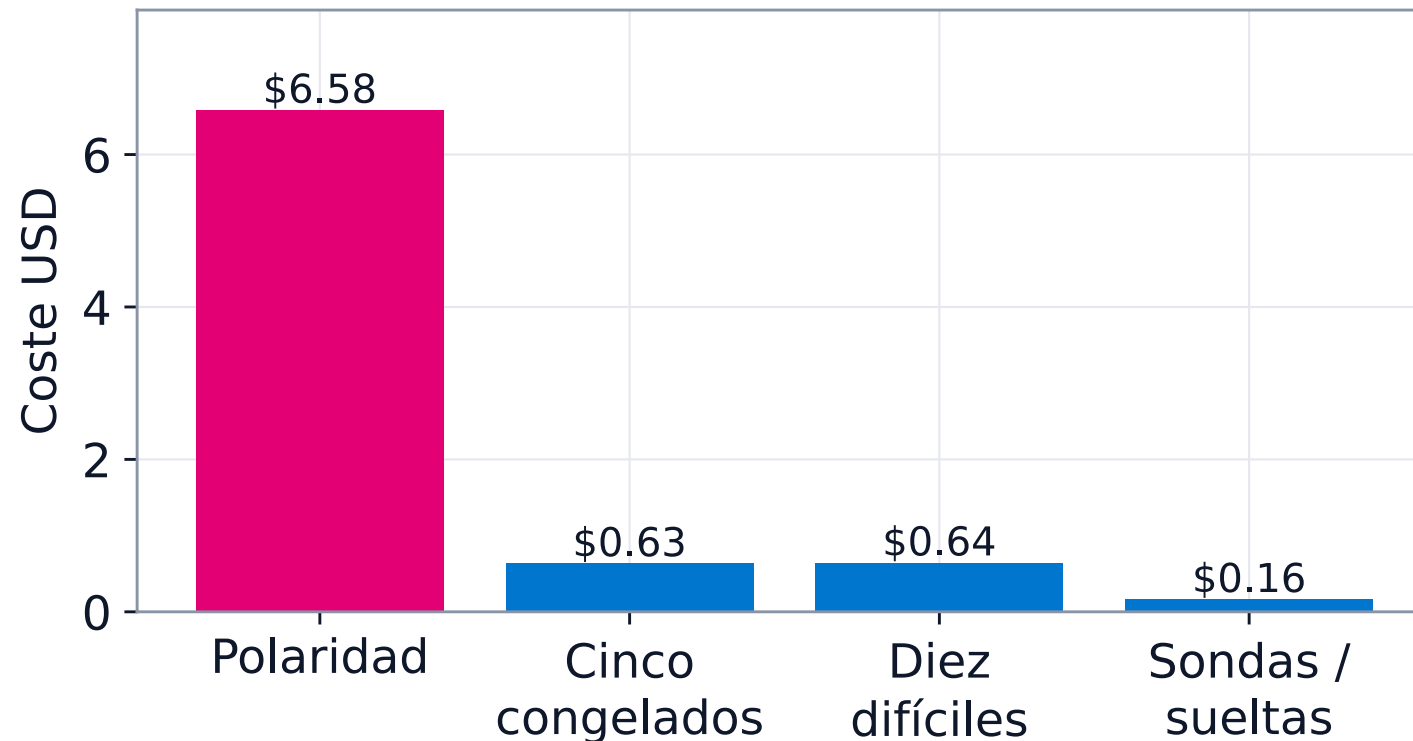
3 AGENTES DETERMINISTAS / GUARDAS

Gobernanza y Verificación Hermética

- **6. Validador estático (Zero-LLM):** Valida sintaxis y esquemas antes de compilar.
- **7. Sandbox Docker:** Ejecuta mvn test -o en contenedor sellado sin red.
- **8. Reparación reflexiva:** Analiza traza de error y reintenta con límite estricto de **máx. 3 iteraciones** antes de escalar a humano (BLOQUEADO).

4. Métricas, Calidad y Costes de Producción.

De «no podía verificar su propia salida» a un pipeline de \$0.06 por microservicio verificado y auditable.



\$0.06 USD

Por microservicio individual generado y verificado (5 etapas LLM).

\$0.20 USD

Con selección de 3 candidatos compitiendo; se elige el que compila y pasa pruebas.

ESTADO ACTUAL DE VERIFICACIÓN

- **100% compilación verificada:** Primera build pasando en sandbox hermético sin red.
- **Paridad de rutas:** Flujo secuencial y grafo LangGraph comparten el mismo estándar.
- **Consumo de tokens:** 78K tokens por servicio; 2.2M ahorrados desde caché.
- **Trazabilidad MLflow:** Registro forense de cada llamada, latencia y coste en SQLite.
- **Punto abierto:** 33% intermitencia bajo carga por reetiquetado SELinux :Z (en resolución).

5. Próximos Pasos: Auto-Mejora con SkillOpt (SOTA).

Aprender de los diagnósticos de fallo para refinar automáticamente las políticas de generación bajo 4 controles formales de fiabilidad.

PROPUESTA SOTA EN ESPACIO DE TEXTO

Los 4 Controles de Fiabilidad

- **1. Presupuesto acotado ($\leq L_t = 4$):** Decaimiento coseno de ediciones permitidas; previene reescrituras caóticas.
- **2. Held-out Validation Gate:** Solo se acepta una regla si mejora estrictamente en tareas no vistas (empates rechazados).
- **3. Búfer de rechazos:** Propuestas fallidas se realimentan como contraejemplos negativos para evitar ciclos.
- **4. Actualización lenta por época:** Consolida aprendizajes globales; sin ella, el sistema **colapsa en -22.5 pts.**
- **Evidencia SOTA:** 52 de 52 celdas ganadas/empatadas (+23.5 pts en GPT-5.5, +24.8 pts en Codex).

Paso	Objetivo técnico	Esfuerzo / Coste
1	Verificador determinista: aislamiento SELinux :Z en volumen Docker	Corto (1–2 días)
2	Diagnóstico rico: minería estructurada de fallos en trazas Surefire	Corto
3	Inyección baseline: medir impacto del manual Java/Maven inicial	\$3–\$5 inferencia
4	Bucle cerrado SkillOpt: optimización reflexiva gobernada por los 4 controles	Condicionado a 1-3

Visión de producto: Un sistema que genera, compila y verifica microservicios por \$0.07, y que perfecciona sus propias reglas de forma continua y gobernada.